

计算思维与程序实践 II

选题参考文档

2026 年 5 月 6 日

1 总体原则

1.1 共同要求

所有选题原则上应满足以下基本要求：

- 至少包含 5 个以上自定义类，体现清晰的面向对象设计。
- 至少使用一种自定义数据结构，例如链表、栈、队列、树或图。
- 鼓励使用指针管理对象关系。
- 鼓励使用模板实现通用容器。
- 必须支持文件读写，例如读取配置、保存结果、导出日志或加载数据集。
- 程序应包含基础异常处理或输入合法性检查。
- 小组展示时应说明类图、核心算法、运行结果和分工。

1.2 评分标准

评分项	分值	说明
功能完成度	35	核心功能是否完整，程序是否能稳定运行。
C++ 语言能力	20	是否合理使用类、对象、指针、动态内存、继承、多态、模板等知识。
数据结构与算法	15	是否设计了合适的数据结构，核心算法是否正确。
文件读写与数据管理	10	是否支持数据导入、结果保存、日志输出或配置文件。
展示效果与交互体验	10	命令行交互是否清晰，输出是否易读，展示是否有亮点。
代码规范与报告	10	命名、注释、模块划分、实验报告与小组分工是否清楚。

2 题目一：校园一天生活模拟器

题目一 校园一天生活模拟器：大一新生的一天重开模拟器

题目简介：设计一个命令行文字模拟游戏。玩家扮演一名大一新生，从早上起床开始安排一天的生活。每一个选择都会改变角色状态，例如体力、心情、绩点、金钱、社交值和编程能力。一天结束后，系统根据玩家的行为给出不同结局，例如“卷王结局”“社牛结局”“摆烂结局”等等。

2.1 核心玩法

程序以时间推进为主线。玩家每天从固定时间开始，例如 7:00 起床。系统展示当前时间、地点和角色状态，然后让玩家选择行动。

当前时间：08:00

当前位置：宿舍

体力：80 心情：65 绩点：3.20

金钱：45 社交：20 编程能力：10

你接下来要做什么？

1. 去上高数课
2. 去食堂吃早饭
3. 回床上继续睡觉
4. 打开电脑写 C++ 作业

不同选择会消耗时间并改变属性。例如，上课可能提高绩点但降低体力；参加社团可能提高社交值但减少学习时间；打游戏可能提高心情但降低绩点。

2.2 必做功能

基础功能要求

1. **角色属性系统：**设计玩家类，包含体力、心情、绩点、金钱、社交值、编程能力等属性。
2. **时间推进系统：**每个事件消耗一定时间，时间到达晚上后进入结局判定。
3. **地点系统：**至少包含宿舍、教室、食堂、图书馆、操场、社团活动室等地点。
4. **事件系统：**每个地点包含若干事件，例如上课、吃饭、运动、写作业、参加社团。
5. **选择系统：**玩家输入选项，不同选项触发不同事件。
6. **多结局系统：**根据最终属性判断结局。
7. **存档与读档：**支持保存当前时间、地点和角色属性。

2.3 推荐类设计

```

1 // 下面只给出一种可能的类划分，成员变量和接口需要小组自行补充。
2 class Player {
3 private:
4     // TODO: 设计体力、心情、绩点、金钱等属性
5 };
6
7 class Event {
8 public:
9     virtual void trigger(Player* player) = 0;
10    virtual ~Event() {}
11 };
12
13 // TODO: 自行设计若干事件派生类，例如学习、社团、随机事件等。
14 class StudyEvent : public Event { };
15 class ClubEvent : public Event { };
16 class RandomEvent : public Event { };
17
18 class Location {
19 private:
20     // TODO: 保存地点名称以及该地点可触发的事件列表
21 };
22
23 class Game {
24     // TODO: 保存当前时间、当前位置、地点列表和游戏状态
25     // TODO: 自行设计存档、读档、结局判定和命令处理接口
26 };

```

2.4 分层要求

层级	要求
基础版	完成玩家属性、地点、事件、时间推进和至少 3 个结局。
标准版	支持随机事件、存档读档、至少 6 个地点和 20 个事件。
挑战版	支持剧情配置文件、成就系统、隐藏剧情、多周目继承属性。

3 题目二：MiniOCR 手写数字识别器

题目二 MiniOCR：不用外部库的手写数字识别器

题目简介：设计一个简化版 OCR 系统。学生不使用 OpenCV、PyTorch 或现成机器学习库，而是自己定义小尺寸数字图像，例如 5×7 或 8×8 的字符矩阵。系统将字符图转换为向量，使用感知机、最近邻或简单神经网络识别数字 0 到 9。

3.1 数据表示

可以使用文本文件保存数字图像。例如数字 0 的 5×7 表示如下：

```
01110
10001
10011
10101
11001
10001
01110
label: 0
```

其中，字符 1 表示黑色像素，字符 0 表示白色像素。系统读取文本后，将其转换为矩阵或一维向量。

3.2 必做功能

基础功能要求

1. **数字样本库：**至少设计数字 0 到 9 的标准样本。
2. **数据读取：**从文本文件读取字符矩阵和标签。
3. **噪声生成：**随机翻转少量像素，生成更多训练样本。
4. **特征转换：**将 5×7 或 8×8 图像转换为向量。
5. **分类器实现：**实现最近邻、感知机或单隐层神经网络中的至少一种。
6. **训练与测试：**输出训练准确率和测试准确率。
7. **手动输入识别：**用户可以输入一个字符图，系统输出识别结果。

3.3 推荐类设计

```
1 // 下面只给出核心对象关系，具体数据表示和算法接口需要小组自行设计。
2 template <typename T>
3 class Matrix {
4 private:
```

```

5     T* data;
6     int rows;
7     int cols;
8 public:
9     // TODO: 构造、析构、元素访问、展开为向量等接口
10 };
11
12 class DigitImage {
13 private:
14     Matrix<int> pixels;
15     int label;
16 public:
17     // TODO: 读取字符图、加入噪声、打印图像、返回标签
18 };
19
20 class Dataset {
21 private:
22     LinkedList<DigitImage*> samples;
23 public:
24     // TODO: 数据加载、样本扩增、训练集/测试集划分
25 };
26
27 class Classifier {
28 public:
29     virtual void train(Dataset* trainSet) = 0;
30     virtual int predict(DigitImage* image) = 0;
31     virtual ~Classifier() {}
32 };
33
34 // TODO: 至少实现一种分类器，可选择最近邻、感知机或简单神经网络。
35 class KNNClassifier : public Classifier { };
36 class PerceptronClassifier : public Classifier { };

```

3.4 分层要求

层级	要求
基础版	实现字符图读取、噪声生成和任意一种简单的分类器，数据集不小于 10000 张图片。
标准版	实现感知机或单隐层神经网络，输出训练与测试准确率。
挑战版	支持混淆矩阵、不同噪声强度测试、用户手写输入、分类器对比。

4 题目三：TinyTensor 自动求导引擎

题目三 TinyTensor：从零实现一个迷你自动求导引擎

题目简介：实现一个极简版自动求导系统。用户可以创建变量，执行加法、乘法、矩阵乘法、平方误差、ReLU 等操作。系统在前向计算时记录计算关系，并自动沿计算图反向传播梯度。

4.1 示例程序

```

1  Tensor x(2.0);
2  Tensor w(3.0);
3  Tensor b(1.0);
4
5  Tensor y = w * x + b;
6  Tensor loss = (y - 10.0) * (y - 10.0);
7
8  loss.backward();
9
10 cout << "loss = " << loss.value() << endl;
11 cout << "grad of w = " << w.grad() << endl;
12 cout << "grad of b = " << b.grad() << endl;

```

该程序对应一个简单函数：

$$y = wx + b, \quad L = (y - 10)^2.$$

系统需要自动计算 L 对 w 和 b 的梯度。

4.2 必做功能

基础功能要求

1. **Tensor 类：**保存数值、梯度、形状和父节点信息。
2. **基本运算：**支持加法、减法、乘法、除法。
3. **计算图记录：**每次运算生成新节点，并记录其来源。
4. **反向传播：**调用 `backward()` 后，根据链式法则计算梯度。
5. **梯度下降：**使用梯度更新参数，拟合简单函数 $y = ax + b$ 。
6. **训练日志：**输出每轮训练的损失值和参数变化。

4.3 标准版扩展

在基础版完成后，可以进一步支持：

- 向量或矩阵 `Tensor`。
- 矩阵乘法 `matmul()`。
- 激活函数 `ReLU`、`Sigmoid`。
- 均方误差损失 `MSELoss`。
- 使用 `TinyTensor` 训练一个小型二维分类器。

4.4 推荐类设计

```
1 // 下面只给出计算图的基本结构，具体梯度规则需要小组自行推导和实现。
2 class Tensor;
3
4 class GradFunction {
5 public:
6     virtual void backward(Tensor* output) = 0;
7     virtual ~GradFunction() {}
8 };
9
10 class Tensor {
11 private:
12     double* data;
13     double* grad;
14     int rows;
15     int cols;
16
17     LinkedList<Tensor*> parents;
18     GradFunction* gradFn;
19
20 public:
21     // TODO: 构造函数、析构函数、数据访问、梯度清零
22     // TODO: 重载 +、-、* 等运算，并在运算时记录计算图
23     // TODO: 实现 backward(), 沿计算图反向传播梯度
24 };
25
26 // TODO: 为不同运算设计不同的反向传播规则。
27 class AddBackward : public GradFunction { };
28 class MulBackward : public GradFunction { };
29 class MatMulBackward : public GradFunction { };
```


4.5 分层要求

层级	要求
基础版	只支持标量 Tensor，完成加减乘除和 backward。
标准版	支持向量或矩阵 Tensor，完成 MSE 损失和梯度下降拟合。
挑战版	支持矩阵乘法、ReLU、Sigmoid，并训练一个小型神经网络。

5 题目四：MiniJSON 解析器

题目四 MiniJSON：轻量级 JSON 解析与数据管理系统

题目简介：实现一个轻量级 JSON 解析库，并设计一组命令对 JSON 文件进行读取、访问、格式化输出和保存。系统需要能够识别 JSON 中常见的数据类型，包括数值、字符串、布尔值、空值、对象和列表，并在内存中建立相应的数据结构。

本题不要求完全复刻成熟 JSON 库，也不要求支持所有边界语法。重点在于让同学们理解：如何把一段文本解析成树形数据结构，如何用指针和多态表示不同类型的 JSON 节点，以及如何通过递归完成层级访问和格式化输出。

5.1 核心目标

学生需要设计一个能够读入 JSON 文本的解析器，将文件内容转换为内存中的对象树。随后，系统应提供简单命令，使用户能够打开文件、访问指定层级、查看当前节点、展开打印子层级，并将格式化后的 JSON 内容写入新文件。

该题目的关键不在于写出最完整的 JSON 标准实现，而在于让学生自己思考以下问题：

- 如何识别字符串、数字、布尔值、空值、数组和对象？
- 如何设计一个统一的基类来保存不同类型的 JSON 数据？
- 数组和对象本质上是递归结构，应该如何用指针组织？
- 如何解析类似 `json["family"][2]` 的层级访问命令？
- 如何在输出时控制缩进，使保存结果更加易读？

5.2 必做功能

基础功能要求

1. **文件读取：**支持读取一个 JSON 文件，例如命令 `read file.json`。
2. **基础数据类型：**支持数值 `float`、字符串 `string`、布尔值 `true/false` 和空值 `null`。
3. **复合数据类型：**支持对象 `{}` 和列表 `[]`。
4. **递归解析：**能够解析嵌套对象和嵌套列表。
5. **层级访问：**支持访问指定层级，例如 `json["family"][2]`。
6. **格式化输出：**能够按照缩进格式打印当前节点或整个 JSON。
7. **格式化保存：**支持将当前 JSON 内容写入新文件，例如 `write format.json`。

5.3 提高功能

扩展功能要求

1. **展开打印**: 支持类似 `json["family"][2] all` 的命令, 展开打印当前层级下的所有子内容。
2. **节点修改**: 支持修改某个字段或数组元素的值。
3. **节点创建**: 支持向对象中添加新的键值对, 或向列表中添加新元素。
4. **节点删除**: 支持删除对象中的某个键, 或删除数组中的某个元素。
5. **错误提示**: 能够对常见语法错误给出提示, 例如缺少引号、缺少逗号、括号不匹配等。
6. **命令循环**: 程序运行后进入交互模式, 用户可以连续输入命令完成操作。

5.4 命令设计示例

```
read file.json
print
json["family"][2]
json["family"][2] all
set json["name"] "Li Hua"
add json["family"] "new member"
delete json["family"][0]
write format.json
exit
```

其中, `print` 用于格式化打印当前 JSON; `all` 表示展开所有子层级; `write` 用于将格式化后的内容保存到文件。具体命令格式可以由小组自行设计, 但需要在报告中说明命令语法和处理流程。

5.5 推荐类设计

```
1 // 下面只给出 JSON 树的组织思路, 具体类型、解析细节和命令接口需要小组自行完善。
2 class JsonValue {
3 public:
4     virtual void print(int indent = 0) const = 0;
5     virtual string type() const = 0;
6     virtual ~JsonValue() {}
7 };
8
9 // TODO: 自行设计 JsonNumber、JsonString、JsonBool、JsonNull 等基础类型节点。
10
11 class JsonArray : public JsonValue {
12 private:
```

```
13     LinkedList<JsonValue*> elements;
14 public:
15     // TODO: 添加、访问、删除数组元素，并递归格式化输出
16 };
17
18 class JsonObject : public JsonValue {
19 private:
20     // TODO: 选择合适的数据结构保存 key-value 对
21 public:
22     // TODO: 查找、修改、添加、删除字段，并递归格式化输出
23 };
24
25 class JsonParser {
26 private:
27     string text;
28     int pos;
29 public:
30     // TODO: 设计 parseValue、parseObject、parseArray 等递归解析函数
31     JsonValue* parse(const string& filename);
32 };
33
34 class JsonCommandSystem {
35 private:
36     JsonValue* root;
37 public:
38     // TODO: 设计 read、print、write、set、add、delete 等命令处理流程
39 };
```

5.6 需要自行思考的问题

1. 如果 JSON 文件中出现空格、换行和缩进，解析器应该如何跳过无关字符？
2. 如何区分字符串中的普通字符和结束引号？是否需要处理转义字符？
3. 对象中的键值对应该如何保存？使用链表、动态数组还是其他结构？
4. 数组和对象都可能嵌套自身，如何用递归解决？
5. 用户输入的层级访问命令如何拆分？如何判断访问路径是否合法？
6. 当解析失败时，程序应该直接退出，还是给出错误位置并继续运行？

5.7 分层要求

层级	要求
基础版	支持读取 JSON 文件，解析数值、字符串、布尔值、空值、对象和列表，并能格式化输出。
标准版	支持嵌套结构解析、层级访问、格式化保存和基础错误提示。
挑战版	支持修改、添加、删除节点，支持交互式命令系统，并能处理较复杂的嵌套 JSON。

5.8 限制说明

本题不允许直接调用现成 JSON 解析库。可以使用 C++ 标准库中的 `string`、文件流和基础容器，但鼓励自行实现链表或动态数组来保存对象成员和数组元素。本题的重点是解析过程、树形结构设计、指针管理、递归处理和格式化输出，而不是完整兼容所有 JSON 标准细节。

6 题目五：Matrix Arena 硬件友好的线性代数计算核设计

题目五 Matrix Arena：硬件友好的线性代数计算核设计

题目简介：实现一个小型线性代数运算库，并围绕矩阵乘法等核心算子开展性能挑战。系统不仅要支持矩阵加减、数乘、转置、求范数、矩阵乘法、解线性方程组等基础功能，还需要借鉴 SPIRAL 的思想：对于同一个数学运算，尝试设计多种等价实现，并通过测试选择当前机器上表现较好的版本。

本题不要求实现真正的 SPIRAL 系统，也不要求 GPU、联网、多线程或复杂 SIMD 指令。本题的重点不是单纯追求最快运行时间，而是在保证计算结果正确的前提下，比较不同矩阵乘法实现方式的性能差异。任何优化版本必须先通过统一正确性测试，才能进入最终 benchmark 排名；若某个版本结果错误，则其运行时间不得作为性能结论依据。

6.1 核心目标

学生需要设计一个 `Matrix<T>` 模板类，并在其基础上实现常用线性代数操作。随后围绕矩阵乘法实现多个不同版本的计算核，例如普通三重循环、不同循环顺序、基于分块的矩阵乘法等。系统需要先对每个计算核进行统一正确性验证，再对通过验证的版本进行性能测试，并给出当前机器上的较优正确实现。

最终，学生需要完成一个基于矩阵运算的小应用或性能实验，用来证明自己的运算核不仅“算得对”，而且能够在正确性可验证的前提下进行性能分析。

6.2 必做功能

基础功能要求

- 模板矩阵类：**实现 `Matrix<T>`，支持 `int`、`double` 等常见类型。
- 动态内存管理：**使用动态内存保存矩阵数据，鼓励优先使用一维连续数组存储矩阵。
- 基本运算：**支持矩阵加法、减法、数乘、矩阵乘法和转置。
- 向量操作：**支持点积、范数、归一化和余弦相似度。
- 线性方程求解：**实现一种基础线性方程组求解方法，例如高斯消元法。
- 文件读写：**支持从 CSV 或 TXT 文件读取矩阵，并将计算结果保存到文件。
- 参考实现：**必须实现一个清晰可靠的朴素矩阵乘法版本，例如 `naive_ijk`，作为后续优化版本的正确性参考。
- 应用插件：**至少实现一个基于矩阵运算的应用，例如线性回归、PageRank、图像卷积、推荐系统或多项式拟合。

6.3 正确性验证要求

正确性优先原则

1. **先验证再测速**：所有优化版本，例如 `ikj`、`blocked` 等，都必须先与参考实现输出进行比较。
2. **正确性闸门**：若某个计算核无法通过正确性测试，则该计算核不得进入性能测试和最终排名。
3. **固定随机种子**：若使用随机矩阵测试，必须固定随机种子，保证不同计算核使用完全相同的输入数据。
4. **边界规模测试**：测试矩阵尺寸不能只选择 512×512 ，还必须包含非方阵和不能被常见 `block size` 整除的尺寸。
5. **误差记录**：对于 `double` 类型矩阵，不要求逐元素完全相等，但必须输出最大绝对误差和相对误差。
6. **结论约束**：任何性能结论都必须建立在正确性验证通过的基础上。错误计算核即使运行时间更短，也不能说明该算法更优。

6.4 正确性测试

固定测试样例

学生至少需要使用以下测试规模验证矩阵乘法实现：

1. 1×1 乘以 1×1 ，用于检查最小规模。
2. 2×3 乘以 3×4 ，用于检查非方阵乘法。
3. 8×8 乘以 8×8 ，用于检查小规模方阵。
4. 31×29 乘以 29×37 ，用于检查非整齐尺寸和分块边界。
5. 64×64 乘以 64×64 ，用于基础性能测试。
6. 255×257 乘以 257×129 ，用于检查无法被常见 `block size` 整除的情况。
7. 512×512 乘以 512×512 ，用于中等规模性能比较。

若进行更大规模测试，例如 1024×1024 或 2048×2048 ，仍然必须先通过正确性验证。

6.5 结果比较标准

对于整数矩阵，要求不同实现的输出结果完全一致。对于浮点矩阵，允许微小数值误差，但必须满足以下条件之一：

$$\max_{i,j} |C_{\text{ref}}(i,j) - C_{\text{test}}(i,j)| < 10^{-8},$$

或者

$$\frac{\|C_{\text{ref}} - C_{\text{test}}\|_F}{\|C_{\text{ref}}\|_F + 10^{-12}} < 10^{-6}.$$

其中, C_{ref} 表示参考实现的结果, C_{test} 表示待测试计算核的结果, $\|\cdot\|_F$ 表示 Frobenius 范数。对于特别大的矩阵, 若完整参考计算代价过高, 可以使用随机向量校验方法作为补充, 例如比较 $C_{\text{test}}r$ 与 $A(Br)$ 是否一致。

6.6 提高功能

性能挑战要求

1. **多版本矩阵乘法:** 至少实现两种不同循环顺序的矩阵乘法, 例如 `ijk`、`ikj` 或其他排列。
2. **连续内存优化:** 比较一维连续数组和二维指针数组在矩阵访问上的差异, 并说明自己的选择。
3. **分块矩阵乘法:** 实现基于 `cache blocking` 思想的矩阵乘法。
4. **分块大小测试:** 支持多个 `block size` 的性能测试, 例如 4、8、16、32、64 等。
5. **自动计时:** 只对通过正确性测试的计算核自动记录运行时间。
6. **重复实验:** 每个通过正确性测试的计算核至少重复运行 3 次, 取平均值或中位数作为最终时间。
7. **自动选择:** 只在通过正确性测试的计算核中, 选择当前机器上运行时间较短的版本。
8. **性能报告:** 输出矩阵规模、实现方式、正确性结果、最大误差、相对误差、运行时间和最快正确版本。

6.7 需要自行思考的问题

1. 对于矩阵乘法, 除了直接三重循环, 还能设计出哪些等价实现?
2. 为什么改变循环顺序可能影响运行时间?
3. 为什么矩阵分块可能提升性能? 分块大小是否越大越好?
4. 不同矩阵规模下, 最快的正确实现是否相同? 请用实验数据说明。
5. 编译选项 `-O0`、`-O2`、`-O3` 是否会影响运行时间? 如果影响, 原因可能是什么?
6. 如果只能优化一个部分, 应该优先优化矩阵乘法、文件读写, 还是其他操作? 为什么?
7. 你的运算核更适合哪类应用? 例如小矩阵频繁计算、大矩阵乘法、推荐系统还是图像卷积?
8. 如果某个版本运行速度非常快但无法通过正确性测试, 应该如何在报告中说明?

6.8 推荐类设计

```

1 // Matrix<T>: 通用矩阵模板, 负责矩阵数据存储、维度管理、基本运算和文件读写。
2 template <typename T>
3 class Matrix {
4 private:
5     T* data;          // 鼓励使用一维连续数组
6     int rows;
7     int cols;
8
9 public:
10    Matrix(int r, int c);
11    Matrix(const Matrix<T>& other);
12    ~Matrix();
13
14    Matrix<T>& operator=(const Matrix<T>& other);
15
16    T& at(int row, int col);
17    const T& at(int row, int col) const;
18
19    int rowCount() const;
20    int colCount() const;
21
22    Matrix<T> operator+(const Matrix<T>& other) const;
23    Matrix<T> operator-(const Matrix<T>& other) const;
24    Matrix<T> scalarMultiply(T k) const;
25    Matrix<T> transpose() const;
26
27    void loadFromFile(const string& filename);
28    void saveToFile(const string& filename) const;
29 };
30
31 // MatMulKernel<T>: 矩阵乘法计算核的抽象基类, 用统一接口封装不同乘法实现。
32 template <typename T>
33 class MatMulKernel {
34 public:
35     virtual Matrix<T> multiply(
36         const Matrix<T>& A,
37         const Matrix<T>& B
38     ) = 0;
39
40     virtual string name() const = 0;
41     virtual ~MatMulKernel() {}
42 };
43
44 // NaiveIJKKernel<T>: 最朴素的 ijk 三重循环乘法, 主要作为正确性参考实现。

```

```

45 template <typename T>
46 class NaiveIJKKernel : public MatMulKernel<T> {
47 public:
48     Matrix<T> multiply(
49         const Matrix<T>& A,
50         const Matrix<T>& B
51     ) override;
52
53     string name() const override;
54 };
55
56 // NaiveIKJKernel<T>: 调整循环顺序的矩阵乘法, 用于比较内存访问顺序对性能的影响。
57 template <typename T>
58 class NaiveIKJKernel : public MatMulKernel<T> {
59 public:
60     Matrix<T> multiply(
61         const Matrix<T>& A,
62         const Matrix<T>& B
63     ) override;
64
65     string name() const override;
66 };
67
68 // BlockedKernel<T>: 基于分块思想的矩阵乘法计算核, 用 blockSize 控制缓存友好粒度。
69 template <typename T>
70 class BlockedKernel : public MatMulKernel<T> {
71 private:
72     int blockSize;
73
74 public:
75     BlockedKernel(int blockSize);
76
77     Matrix<T> multiply(
78         const Matrix<T>& A,
79         const Matrix<T>& B
80     ) override;
81
82     string name() const override;
83 };
84
85 // MatrixComparator<T>: 矩阵结果比较工具, 用于判断不同计算核输出是否一致并统计误差。
86 template <typename T>

```

```

87 class MatrixComparator {
88 public:
89     static bool exactEqual(
90         const Matrix<T>& A,
91         const Matrix<T>& B
92     );
93
94     static bool almostEqual(
95         const Matrix<double>& A,
96         const Matrix<double>& B,
97         double absTol = 1e-8,
98         double relTol = 1e-6
99     );
100
101     static double maxAbsError(
102         const Matrix<double>& A,
103         const Matrix<double>& B
104     );
105
106     static double relativeFrobeniusError(
107         const Matrix<double>& A,
108         const Matrix<double>& B
109     );
110 };
111
112 // KernelTestSuite<T>: 计算核测试套件, 负责用参考实现验证候选计算核的正确性。
113 template <typename T>
114 class KernelTestSuite {
115 private:
116     MatMulKernel<T>* referenceKernel;
117
118 public:
119     KernelTestSuite(MatMulKernel<T>* ref);
120
121     bool testKernel(
122         MatMulKernel<T>* kernel,
123         const Matrix<T>& A,
124         const Matrix<T>& B
125     );
126
127     void runAllTests(
128         LinkedList<MatMulKernel<T>*>& kernels
129     );
130 };
131

```

```
132 // KernelAutoTuner<T>: 计算核自动调优器, 负责登记候选实现、运行 benchmark 并
    选择最快正确版本。
133 template <typename T>
134 class KernelAutoTuner {
135 private:
136     LinkedList<MatMulKernel<T>*> kernels;
137
138 public:
139     void addKernel(MatMulKernel<T>* kernel);
140
141     MatMulKernel<T>* findBestValidKernel(
142         const Matrix<T>& A,
143         const Matrix<T>& B
144     );
145
146     void runBenchmark(
147         const Matrix<T>& A,
148         const Matrix<T>& B
149     );
150 };
151
152 // LinearSolver: 线性方程组求解工具类, 提供高斯消元等基于矩阵的数值方法。
153 class LinearSolver {
154 public:
155     static Matrix<double> solveGaussian(
156         Matrix<double> A,
157         Matrix<double> b
158     );
159 };
160
161 // Application: 矩阵应用插件的抽象基类, 用统一 run() 接口组织不同应用任务。
162 class Application {
163 public:
164     virtual void run() = 0;
165     virtual ~Application() {}
166 };
167
168 // LinearRegressionApp: 线性回归应用插件, 用自定义矩阵运算完成参数拟合与预
    测。
169 class LinearRegressionApp : public Application {
170 public:
171     void run() override;
172 };
173
174 // PageRankApp: PageRank 应用插件, 用矩阵迭代计算节点重要性或影响力排名。
```

```
175 class PageRankApp : public Application {
176 public:
177     void run() override;
178 };
```

6.9 建议输出示例

Benchmark 输出示例

Matrix size: 512 x 512
Random seed: 2026

Correctness check:
[1] naive_ijk reference
[2] naive_ikj PASS max_error = 0.00e+00 rel_error = 0.00e+00
[3] blocked_8 PASS max_error = 1.42e-10 rel_error = 3.18e-13
[4] blocked_16 PASS max_error = 1.37e-10 rel_error = 2.95e-13
[5] blocked_32 PASS max_error = 1.50e-10 rel_error = 3.21e-13
[6] blocked_64 FAIL max_error = 2.18e+03 rel_error = 4.77e-01

Kernel benchmark:
[1] naive_ijk time = 1.842 s
[2] naive_ikj time = 0.936 s
[3] blocked_8 time = 0.718 s
[4] blocked_16 time = 0.503 s
[5] blocked_32 time = 0.431 s

Best valid kernel: blocked_32
Invalid kernels are excluded from ranking.

6.10 分层要求

层级	要求
基础版	实现 Matrix<T>、矩阵加减乘、数乘、转置、文件读写，并通过小规模和非方阵正确性测试。
标准版	实现向量范数、余弦相似度、高斯消元，并完成至少一个应用插件；所有核心运算需要给出测试样例。
挑战版 1	实现多版本矩阵乘法和分块矩阵乘法，但每个版本必须先通过统一正确性测试。
挑战版 2	在通过正确性测试的计算核中比较不同矩阵规模、不同 block size、不同编译选项下的性能差异，并解释原因。

6.11 限制说明

本题不要求调用外部线性代数库，不要求实现完整 BLAS，不要求使用 GPU，不要求实现真正的 SPIRAL 代码生成系统，也不要求学生手写复杂 SIMD 指令。本题重点是通过 C++ 实现一个可测试、可比较、可解释的小型线性代数运算核，并理解软件实现方式与硬件运行效率之间的关系。本题的最终结论必须建立在正确性验证通过的基础上；未通过正确性测试的计算核不得参与最终性能排名。